

OOP'S CONCEPT IN JAVA

- **Object-oriented programming System(OOPs)** is a programming concept that is based on “**objects**”.
- It allows users to create objects they want and create methods to handle those objects.
- The basic concept of OOPs is to create objects, re-use them throughout the program, and manipulate these objects to get results.
- The main principles of object-oriented programming are abstraction, encapsulation, inheritance, and polymorphism. These concepts aim to implement real-world entities in programs.

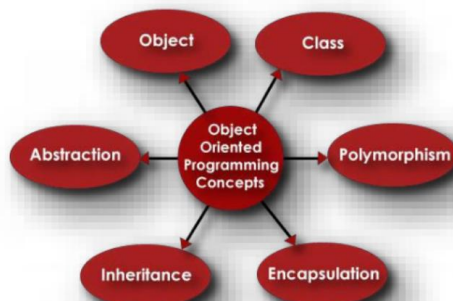
OOPs (Object-Oriented Programming System)

It simplifies software development and maintenance by providing some concepts:

OOPs (Object-Oriented Programming System)

It simplifies software development and maintenance by providing some concepts:

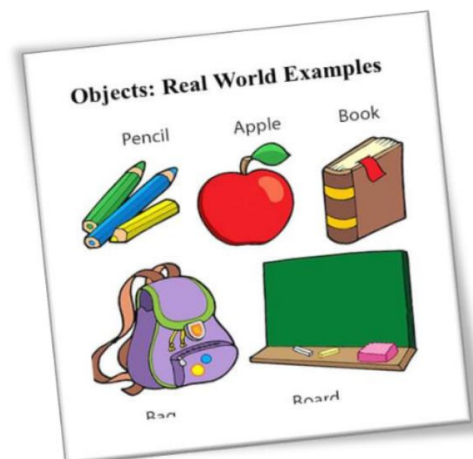
- ✚ **Object**
- ✚ **Class**
- ✚ **Inheritance**
- ✚ **Polymorphism**
- ✚ **Abstraction**
- ✚ **Encapsulation**



❖ Object

- Any entity that has state and behavior is known as an object.
- For example, a chair, pen, table, keyboard, bike, etc. It can be physical or logical.
- An Object can be defined as an instance of a class.
- An object contains an address and takes up some space in memory.
- Objects can communicate without knowing the details of each other's data or code.
- The only necessary thing is the type of message accepted and the type of response returned by the objects.

Example: A dog is an object because it has states like color, name, breed, etc. as well as behaviors like wagging the tail, barking, eating, etc.



❖ Class

- Collection of objects is called class. It is a logical entity.
- A class can also be defined as a blueprint from which you can create an individual object. Class doesn't consume any space.

❖ Inheritance

- When one object acquires all the properties and behaviors of a parent object, it is known as inheritance.
- It provides code reusability.
- It is used to achieve runtime polymorphism.

❖ Polymorphism

- If one task is performed in different ways, it is known as polymorphism. For example: to convince the customer differently, to draw something, for example, shape, triangle, rectangle, etc.
- In Java, we use method overloading and method overriding to achieve polymorphism.
- Another example can be to speak something; for example, a cat speaks meow, dog barks woof, etc.

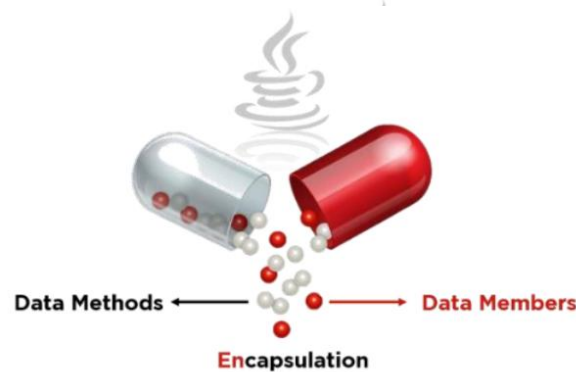


❖ Abstraction

- Hiding internal details and showing functionality is known as abstraction. For example phone call, we don't know the internal processing.
- In Java, we use abstract class and interface to achieve abstraction.

❖ Encapsulation

- Binding (or wrapping) code and data together into a single unit are known as encapsulation. For example, a capsule, it is wrapped with different medicines.
- A java class is the example of encapsulation. Java bean is the fully encapsulated class because all the data members are private here.



❖ Coupling

- Coupling refers to the knowledge or information or dependency of another class.
- It arises when classes are aware of each other.
- If a class has the details information of another class, there is strong coupling. In Java, we use private, protected, and public modifiers to display the visibility level of a class, method, and field.
- You can use interfaces for the weaker coupling because there is no concrete implementation.

❖ Cohesion

- Cohesion refers to the level of a component which performs a single well-defined task.
- A single well-defined task is done by a highly cohesive method.
- The weakly cohesive method will split the task into separate parts.
- The java.io package is a highly cohesive package because it has I/O related classes and interface.
- However, the java.util package is a weakly cohesive package because it has unrelated classes and interfaces.

❖ Association

Association represents the relationship between the objects. Here, one object can be associated with one object or many objects. There can be **four types** of association between the objects:

- **One to One**
- **One to Many**
- **Many to One, and**
- **Many to Many**

❖ Aggregation

- Aggregation is a way to achieve Association.
- Aggregation represents the relationship where one object contains other objects as a part of its state.
- It represents the weak relationship between objects.
- It is also termed as a has-a relationship in Java.
- Like, inheritance represents the is-a relationship. It is another way to reuse objects.

CLASSES AND OBJECT

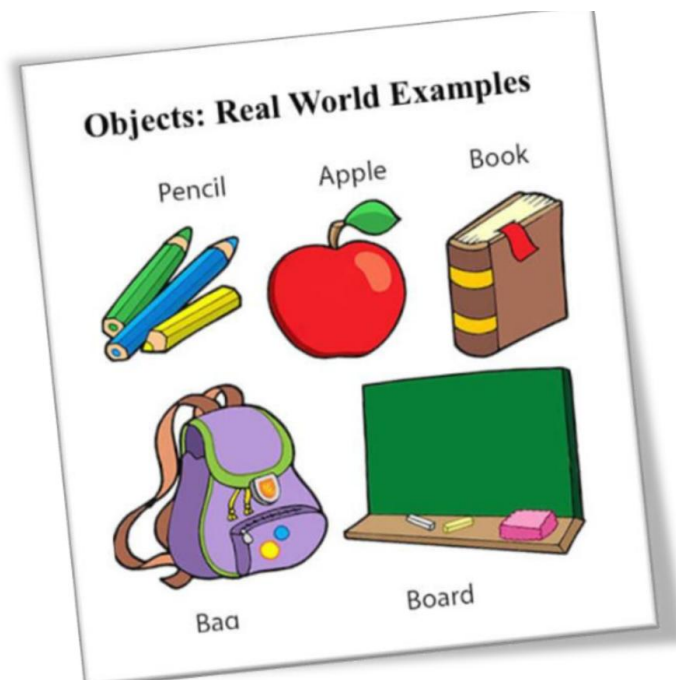
❖ Java Object

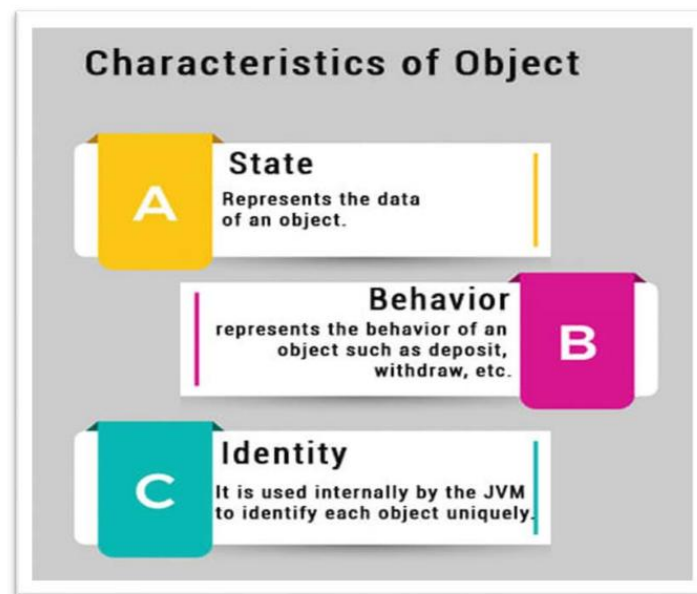
An **object** in Java is a basic unit of Object-Oriented Programming and represents real-life entities. **Objects** are the instances of a class that are created to use the attributes and methods of a class. A typical Java program creates many objects, which as you know, interact by invoking methods.

An object consists of :

- **State:** It is represented by attributes of an object. It also reflects the properties of an object.
- **Behavior:** It is represented by the methods of an object. It also reflects the response of an object with other objects.
- **Identity:** It gives a unique name to an object and enables one object to interact with other objects.

Example of an object: dog





Initializing a Java object:

The new operator instantiates a class by allocating memory for a new object and returning a reference to that memory. The new operator also invokes the class constructor.

Example:

```
// Class Declaration
public class Dog {
    // Instance Variables
    String name;
    String breed;
    int age;
    String color;

    // Constructor Declaration of Class
    public Dog(String name, String breed, int age, String color) {
        this.name = name;
        this.breed = breed;
        this.age = age;
        this.color = color;
    }

    // Method 1: Getter for name
    public String getName() {
        return name;
    }
}
```

```
// Method 3: Getter for age
public int getAge() {
    return age;
}

// Method 4: Getter for color
public String getColor() {
    return color;
}

// Overridden toString() method
@Override
public String toString() {
    return "Hi, my name is " + this.getName() +
        ".\nMy breed, age, and color are " +
        this.getBreed() + ", " + this.getAge() +
        ", " + this.getColor();
}

// Main method
public static void main(String[] args) {
    // Creating an instance of Dog with specific values
    Dog tuffy = new Dog("tuffy", "papillon", 5, "white");

    // Calling toString() method and printing the result
    System.out.println(tuffy.toString());
}
}
```

Output:

```
Hi, my name is tuffy.
My breed, age, and color are papillon, 5, white
```

Initialize by using **method/function**:

```
public class Toppeworld {  
    // Static variables to store software information  
    static String sw_name;  
    static float sw_price;  
  
    // Static method to set software information  
    static void set(String n, float p) {  
        sw_name = n;  
        sw_price = p;  
    }  
  
    // Static method to get and print software information  
    static void get() {  
        System.out.println("Software name is: " + sw_name);  
        System.out.println("Software price is: " + sw_price);  
    }  
  
    // Main method  
    public static void main(String args[]) {  
        // Setting software information using the set() method  
        Toppeworld.set("Visual Studio", 0.0f);  
  
        // Getting and printing software information using the get() method  
        Toppeworld.get();  
    }  
}
```

Output:

```
Software name is: Visual Studio  
Software price is: 0.0
```

❖ Java Classes

A **class** in Java is a set of objects which shares common characteristics/ behavior and common properties/ attributes. It is a user-defined **blueprint** or **prototype** from which objects are created. For example, Student is a class while a particular student named Ravi is an object.

Properties of Java Classes:

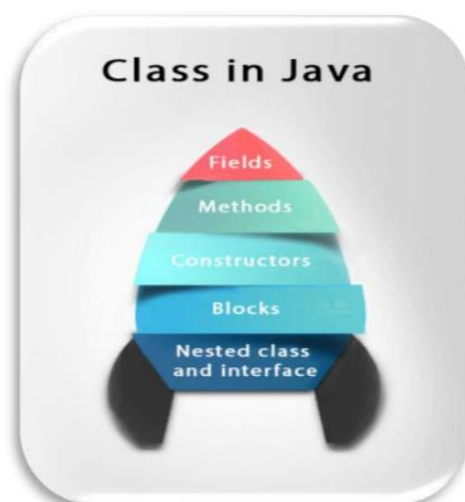
1. Class is not a **real-world** entity. It is just a **template** or **blueprint** or prototype from which objects are created.
2. Class does not occupy memory.
3. Class is a group of variables of different data types and a group of methods.

4. **A Class in Java can contain:**

- Data member
- Method
- Constructor
- Nested Class
- Interface

Syntax to declare a class:

```
class <class_name>{  
  
    field;  
  
    method;  
  
}
```



Example of Java Class:

```
// Java Program for class example

class Student {
    // data member (also instance variable)
    int id;
    // data member (also instance variable)
    String name;

    public static void main(String args[]) {
        // creating an object of Student
        Student s1 = new Student();

        // Accessing and printing the default values of instance variables
        System.out.println(s1.id);    // Default value for int (0)
        System.out.println(s1.name);  // Default value for String (null)
    }
}
```

Output:

```
0
null
```

Components of Java Classes:

In general, class declarations can include these components, in order:

1. **Modifiers:** A class can be public or has default access (Refer this for details).
2. **Class keyword:** class keyword is used to create a class.
3. **Class name:** The name should begin with an initial letter (capitalized by convention).

4. **Superclass(if any):** The name of the class's parent (superclass), if any, preceded by the keyword `extends`. A class can only extend (subclass) one parent.
5. **Interfaces(if any):** A comma-separated list of interfaces implemented by the class, if any, preceded by the keyword `implements`. A class can implement more than one interface.
6. **Body:** The class body is surrounded by braces, `{ }`.

Difference Between **Class** and **Object**:

Class	Object
A class is a template for creating objects in program.	The object is an instance of a class.
A class is a logical entity	Object is a physical entity
A class does not allocate memory space when it is created.	Object allocates memory space whenever they are created.
You can declare class only once.	You can create more than one object using a class.
Example: Car.	Example: Jaguar, BMW, Tesla, etc.
Class generates objects	Objects provide life to the class.
Classes can't be manipulated as they are not available in memory.	They can be manipulated.

It doesn't have any values which are associated with the fields.

Each and every object has its own values, which are associated with the fields.

You can create class using "class" keyword.

You can create object using "**new**" keyword in Java